

COMP20240 Relational Database Assignment

Description on Databases

I have a total of 7 tables in my database all with the purpose to support hospitals trying to recruit for different positions and to help manage all the information that goes with the interviews. The tables store information about the particular hospitals recruiting, the jobs being offered, candidates that have applied and their skills, and the information regarding the interviews and results.

- Hospital_info: gives basic information about the hospitals that have open positions requiring specific skills; the primary key, PK, is the hospital id. With the additional attributes of hospital_name, address, and phone_number.
- Candidate_info: gives basic details about each candidate applying for a position. The PK is candidate_id. With the additional attributes of firstname, lastname, address, and tel_number
- Skill_for_job: gives a list of skills that candidates can have, and positions can require. The PK is skill_id. With an additional attribute of skill_name to help the organisation of the skills to be a bit cleaner.
- Candi_skills: gives details of skills each candidate possesses this is the many to many relationship between candidate and skills. Both attributes, candidate_id and skill_id, are the PK and both are also Foreign Keys, FK, in respect to the relation of candidat_details, and skill_for_job.
- Position_needed: this table goes into the various positions needed from hospitals and the required skills for the position. The position_id is the pk and the fk is hospital_id in respect to the relation of Hospital_info. There is an additional attribute called position_name.
- Position_skill_needed: gives the skills needed for a given position it allows for a many to many relationship between positions and skills. Both attributes, position_id and skills_id, are PK and FK in respect to the relations of position_needed and skill_for_job

Assumptions and Additions and Reaction Policies:

Assumptions:

Different hospitals advertising the same job openings may require different skill sets. This assumption is a bit more practical and acknowledges that each hospital can have specific needs for the same role. For example, one hospital might look for a nurse skilled in emergency care, while another might prioritise a nurse with paediatric expertise. This approach aligns better with a database structure that can support diverse skill requirements for each job-hospital pairing.

Table additions : Due to candidates having multiple skills and each position requiring multiple skills per position that leads to having a many-to-many relationship leading me to add two additional tables to the database. I was able to ensure that the data would not be duplicated and keep the data consistent. This way maintenance of data is also simplified, it is easier to update skills or positions in just one table rather than multiple relations.

Reaction Policies:

In the table **candi_skills**:

I set the fk of candidate_id in candidate_details to on update cascade, that means that if candidate_id changes at all it will be reflected in candi_skills. The same for on delete cascade. If a candidate is deleted in the candidate details table all of their information should also be deleted for candi_skills.

For skill_id I set it to on update cascade and on delete restrict. For update it is cascade because if the skill_id changes in skill_for_jobs, it should update. For delete is set to restrict to prevent deleting a skill if it is associated with any candidate. This allows for data integrity.

In the table **position_skill_needed**:

For position_id I set the reaction policies to on update, cascade, and on deleted, cascade. The reasoning is the same as candi_skills above: if position_id changes then it should be reflected in this table, and if a position is deleted all related information should be updated based on that deletion.

For skill_id I set the policies to on update, cascade, and on delete restrict. This is to update if skill_id is updated in the original table skill_for_job. However, we want to make sure that if a position is deleted, the relevant skills are retained.

In the table **position_needed**:

For hospital_id I set the policies to be on update cascade and on delete restrict. The reasoning is the same as above in case the hospital_id changes, that change should be reflected in this table. And it is restricted on deletion to prevent the deletion of a hospital if there are positions linked to it.

In the table **interview table**:

For candidate_id I restrict for both on update and on delete due to maintaining data consistency and preventing deletion of a candidate that ensures that the interview data remains intact.

For position_id the policies are on update, cascade, and on delete, restrict. If position_id is updated it will automatically update in this table. For delete I want to prevent deletion of a position with a related interview, this way the integrity of the interview history is maintained



